Practical - 4

JavaScript DOM Manipulation, Event Handling, and UI Interactivity

Problem Definition: Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.

A. Theory

1. **How are DOM elements selected and modified?**

Answer :
They are selected using document.getElementById('enrollment') and document.body. They are modified by retrieving input values (.value) and updating CSS classes (classList.toggle() and classList.add()).

2. **Are event listeners attached correctly?**

Answer :
They are functional, but it uses a mix of standard listeners (document.addEventListener('DOMContentLoaded')) and inline HTML handlers (onsubmit="validateAuth(...)", onclick="toggleDarkMode()"). While it works, standardizing on addEventListener is generally best practice.

3. **Is localStorage used for remembering theme choice?**

Answer :
Yes. The script uses localStorage.setItem('darkMode', ...) to save the choice and localStorage.getItem('darkMode') on page load to reapply it.

4. **Does interactivity improve usability without breaking accessibility?**

Answer :
It improves usability by offering dark mode and immediate form validation. However, accessibility could be better; using alert() for form errors isn't ideal for screen readers, and replacing it with semantic, inline error messages would improve accessibility.

B. Implementation

Client-Side Validation :- Added JavaScript to validate the Enrollment ID format on the login/register forms using Regular Expressions (Regex)
and event.preventDefault() to stop invalid submissions.

Theme Toggling :- Created a "Dark Mode" toggle that dynamically updates the page theme by adding/removing CSS classes via DOM manipulation (classList.toggle).

State Persistence :- Integrated browser localStorage to save the user's dark mode preference, ensuring the theme remains consistent even after refreshing or navigating across different pages.

C. Evidence of Work (Screenshots)

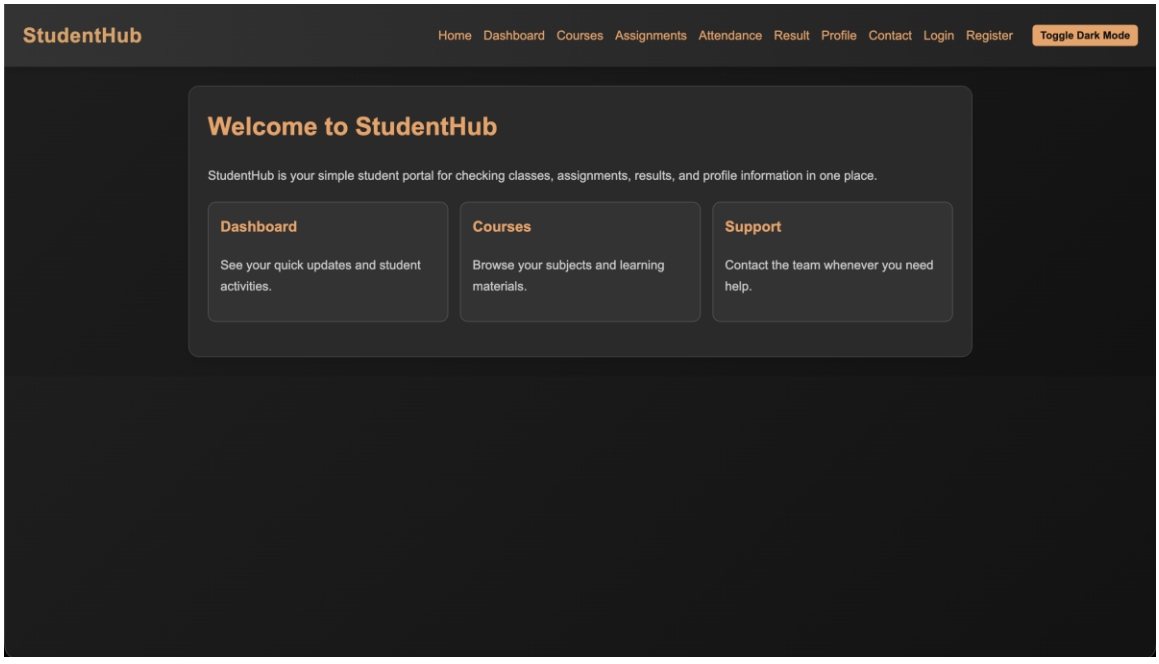


Figure 4.1 :- DARK MODE

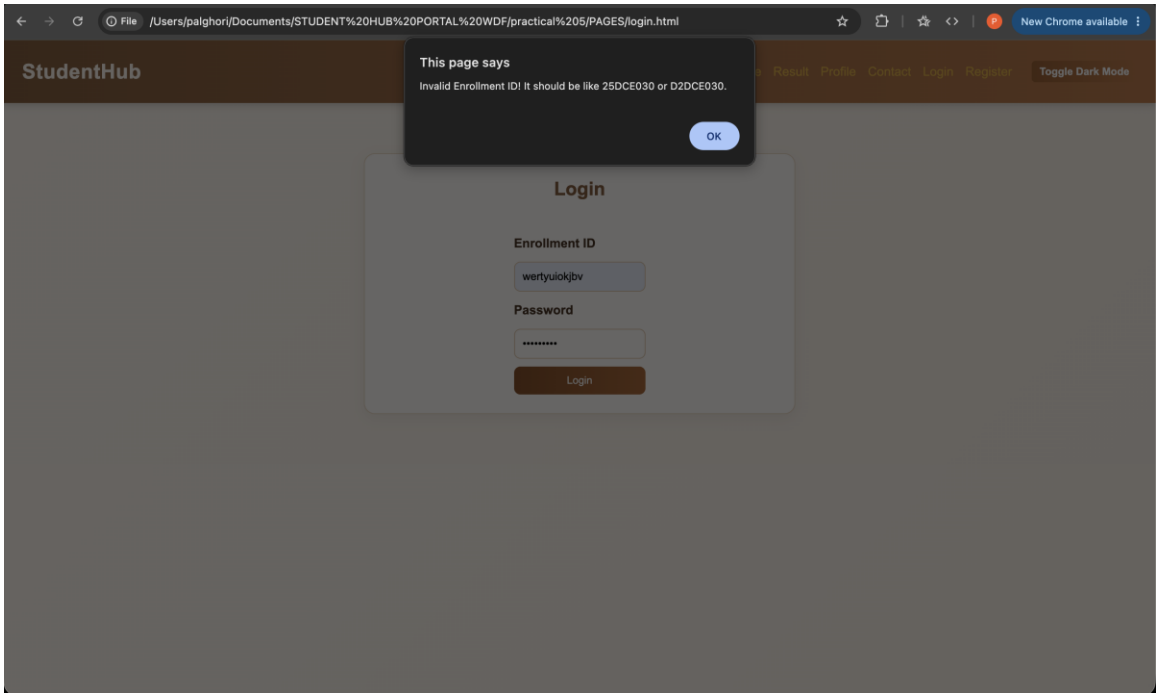


Figure 4.2 :- Regex in Login Form

D. GitHub Repository Link:

https://github.com/palghori/STUDENT-HUB-PORTAL-WDF/tree/main/practical%204

E. Challenges Faced:

Regex Pattern Matching : Constructing the exact Regular Expression (/^\d{2}[a-zA-Z]{3}\d{3}$/) to strictly enforce the specific enrollment ID format without accidentally rejecting valid inputs.

Theme State Reset : Initially, the dark mode would reset when navigating to a new page or refreshing. This required identifying localStorage as the solution to persist the state.

DOM Timing Issues : Ensuring the JavaScript for applying the saved theme only ran after the HTML had fully loaded to prevent errors, which was resolved using the DOMContentLoaded event listener.

F. Learning Outcomes:

DOM Manipulation: Gained hands-on experience selecting HTML elements (getElementById), reading input values (.value), and dynamically altering CSS classes.

Client-Side Security/Validation: Learned how to intercept form submissions and validate user input on the frontend before interacting with a backend.

Browser Storage: Understood how to utilize Web Storage APIs (localStorage) to persistently store user preferences across sessions.

Event Handling: Developed an understanding of how to listen to and manage browser events (like onsubmit, onclick, and DOMContentLoaded).

Faculty Remarks: